МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: П. Д. Косарим
Группа: М8О-307БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: Изучение основ текстурирования в 3D-графике: загрузка изображений из файловой системы, создание текстур, использование текстурных координат вершин и сэмплеров для наложения текстур на объекты с использованием API Vulkan, GLFW и ImGUI.

Метод решения

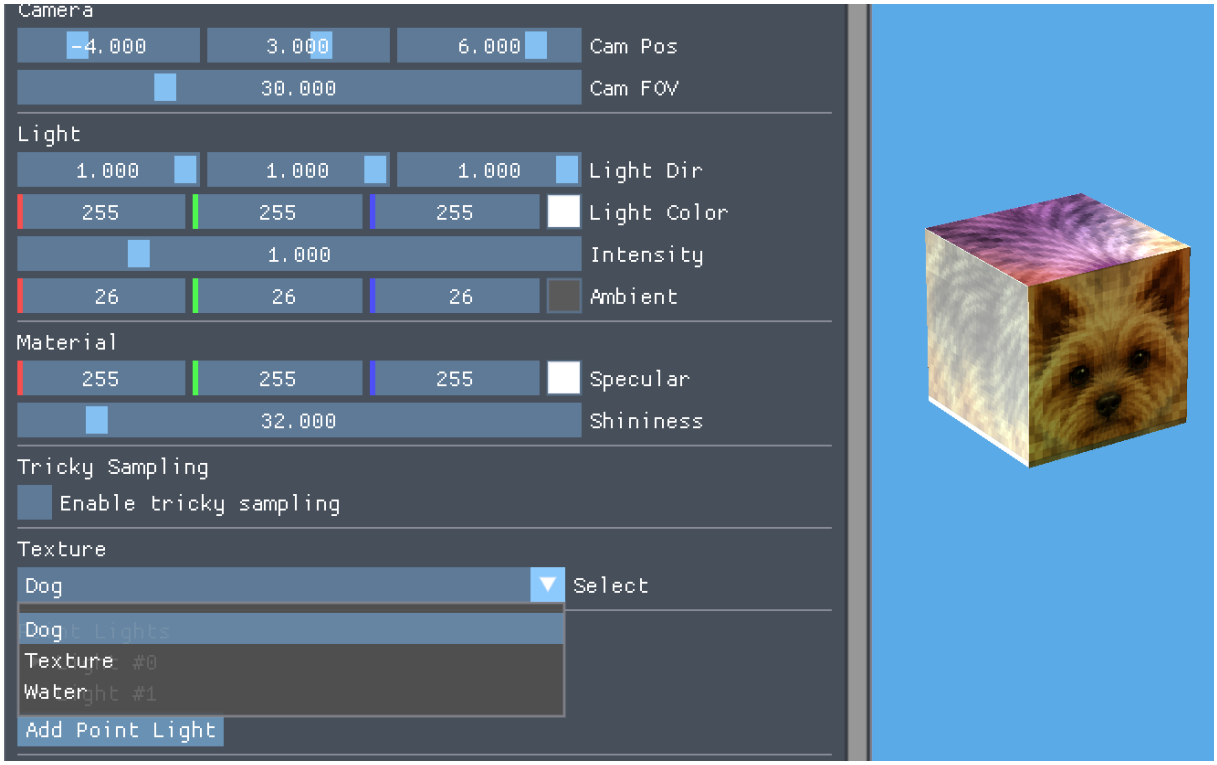
Для реализации текстурирования был использован стартовый код предыдущей лабораторной работы. Были выполнены следующие шаги:

1. Загрузка изображений и создание текстур:
   С помощью библиотеки stb_image.h реализована загрузка изображений из файловой системы (форматы PNG, JPEG). Создан staging-буфер, из которого данные через команды GPU (vkCmdCopyBufferToImage) копируются в память видеокарты (VRAM). Для каждой текстуры создан VkImageView и VkSampler с линейной фильтрацией и режимом наложения (REPEAT).
2. Текстурные координаты и VRAM:
   В структуру вершины Vertex добавлен атрибут UV-координат (location = 3). Для куба координаты сгенерированы процедурно для каждой из 6 граней. Вершинный и индексный буферы переведены в VRAM с использованием промежуточного staging-буфера, как того требует задание.
3. Дескрипторы и мультитекстурирование:
   Создан отдельный набор дескрипторов (Set 2) с VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER. Для каждой загруженной текстуры (Dog, Texture, Water) создан свой VkDescriptorSet. В ImGui добавлен выпадающий список (Combo) для выбора текущей текстуры для модели.
4. Нетривиальное сэмплирование:
   В конвейер (Pipeline Layout) добавлены push-константы (структура PushConstants с полями time и useTricky). Во фрагментном шейдере cube.frag реализовано ветвление: если useTricky != 0, выполняется множественное сэмплирование текстуры по трем разным смещенным UV-координатам, зависящим от времени. Полученные цвета усредняются и модулируются синусоидальной функцией (меняются каналы R, G, B). В ImGui

добавлен чекбокс «Enable tricky sampling» для переключения эффекта.

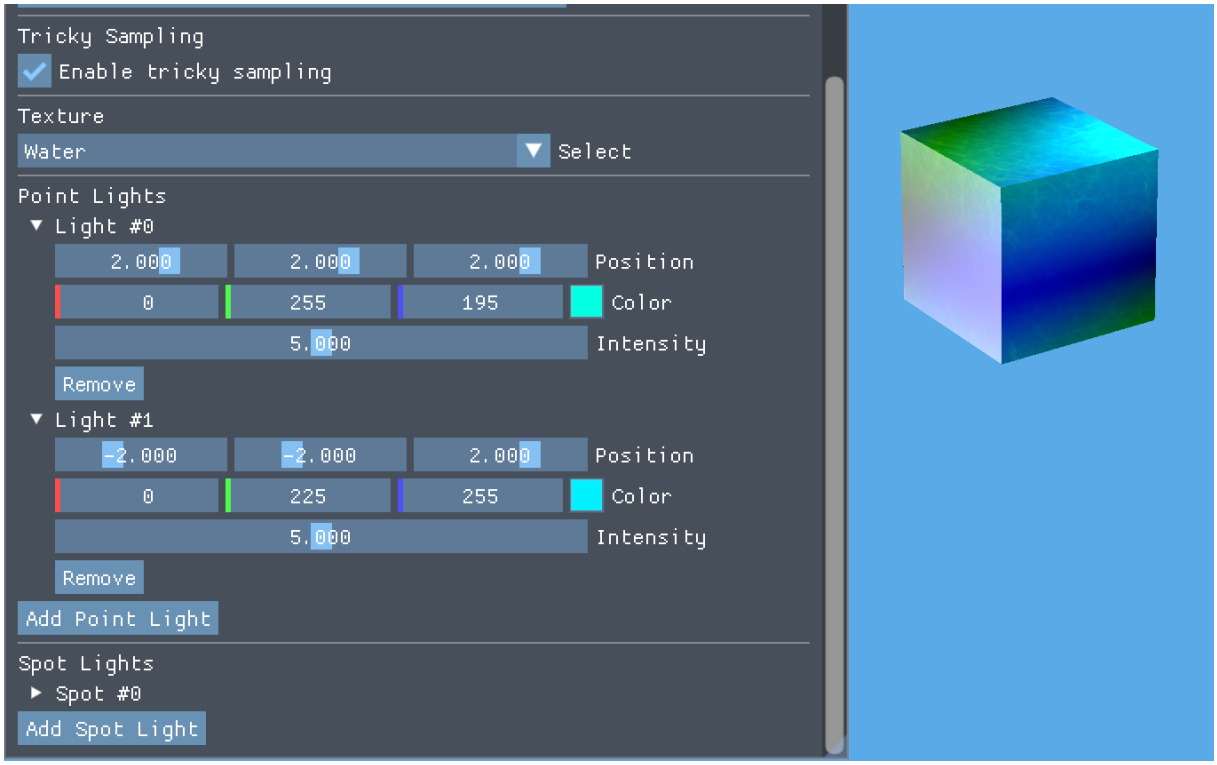
Результаты

Скриншот 1. Базовое текстурирование с выбором текстуры



На рисунке показан результат наложения текстуры на куб. В панели ImGui доступен выбор текстуры из выпадающего списка (Dog, Texture, Water). Реализована модель освещения Блинн-Фонга, которая корректно взаимодействует с текстурой альбедо. Вершинные и индексные данные хранятся в VRAM.

Скриншот 2. Нетривиальное сэмплирование



При включении чекбокса «Enable tricky sampling» в шейдер через push-константы передается флаг и время. Визуально это проявляется в виде динамического переливания цветов и наложения нескольких копий текстуры. Это достигается за счет множественного сэмплирования с разными UV-координатами и модуляции цвета, зависящей от времени.

Выводы

В ходе выполнения лабораторной работы я научилась:

- Загружать изображения из файловой системы и декодировать их с помощью сторонних библиотек.
- Создавать текстуры в Vulkan, используя staging-буферы для переноса данных в VRAM, и настраивать сэмплеры.
- Передавать текстурные координаты через вершины и использовать их для выборки цвета из текстуры (альбедо).
- Организовывать множественные наборы дескрипторов для разных материалов и переключать их в реальном времени через UI.
- Передавать данные в шейдер с помощью push-констант (без использования Uniform-буферов).
- Реализовывать сложные эффекты сэмплирования во фрагментном шейдере, такие как множественная выборка и модуляция цвета, параметризованные от времени.